



**Tecnológico
de Monterrey**

MA. Actividad: Roomba

Mauricio Emilio Monroy González
A01029647

Campus Santa Fe

Modelación de sistemas multiagentes con gráficas computacionales

Octavio Navarro

18 de noviembre del 2025

MA. Actividad: Roomba

Problema que se está resolviendo, y la propuesta de solución.

El reto consiste en desarrollar un ambiente cuadriculado con celdas limpias o sucias, y con agentes que limpian el cuarto al pasar sobre los espacios sucios. Los agentes Roomba tienen una batería, y al generarse llevan consigo una estación de carga en su posición inicial. Al navegar, si encuentra más posiciones de carga las puede recordar, y si se encuentra con más agentes, comunicarse entre sí para transferir las celdas visitadas y estaciones de carga conocidas. Cada movimiento cobrará 1% de la batería del agente, y cuando detecte que está en el punto de que tiene la batería justa para regresar a una estación de carga, calcula el regreso a la estación más cercana. En cada Step dentro de una estación de carga, la batería se regenera en un 5%.

Mi propuesta involucra diseñar a cada agente como un autómata finito con máquina de estados, los cuales involucrarían los siguientes estados:

- Inactivos: sin batería
- Cargándose: en estación de carga y batería menor a 100%
- Regresando a base: batería baja, según determinado a distancia a base
- Explorando: estado default, donde busca moverse a celdas sucias, o si no a celdas desconocidas vecinas, o si no aleatoriamente.

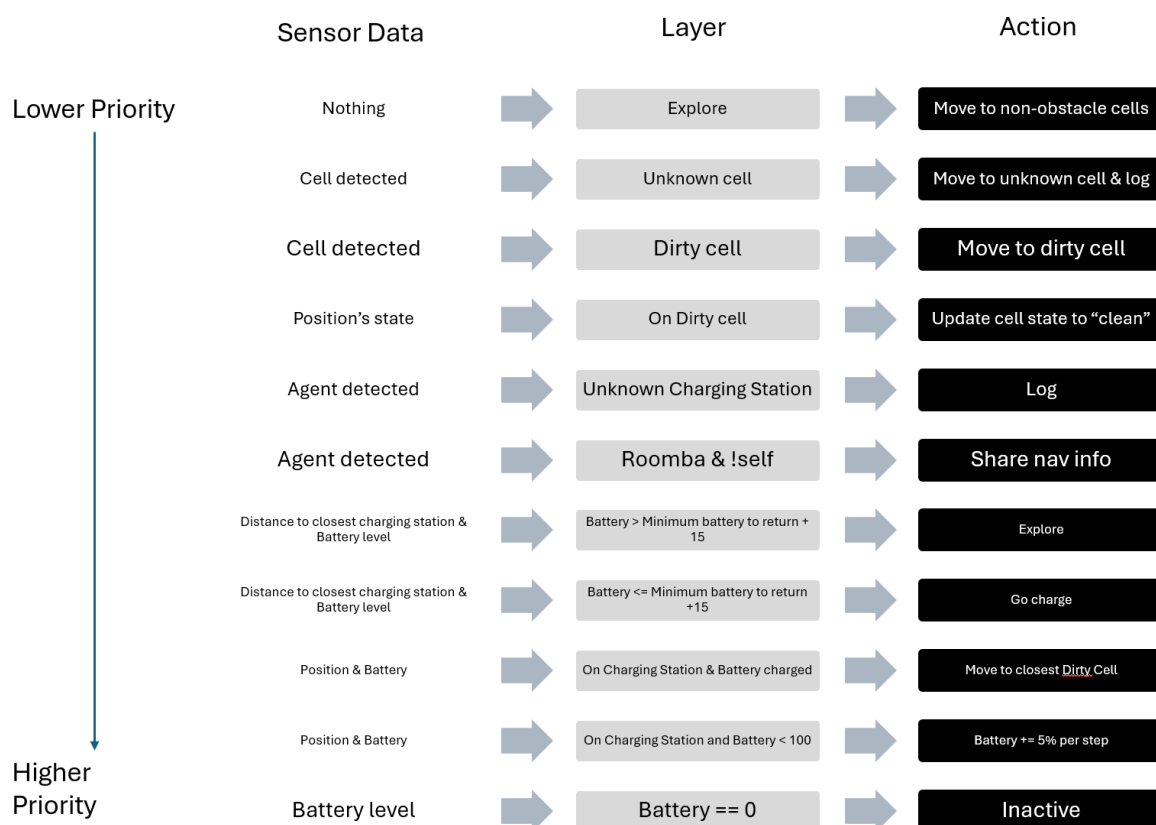
De igual forma, mi configuración realiza los cálculos de batería dejando un 15% de margen adicional, por si se encuentra con algún obstáculo u agente que le estorbe y pueda recalcular con cierta seguridad de que llegará a su destino. Los cálculos de pathfinding los realiza usando Breadth-First-Search. El agente recuerda el estado de las celdas que ha visto, para así después de una carga regresar a la sucia más próxima. En algún punto se intentó instalar un sistema de checkpoints, pero esto evitaba que llegara a lugares nuevos si siempre trataba de volver a donde ya había ido. Incluso hubo una simulación que ni en 10000 steps logró limpiar la habitación, pero en un ambiente colaborativo sí era exitosa la simulación porque se apoyaban de la comunicación para no repetir trayectorias y cada uno ir a donde no conocían. Para solucionar ello, se configuró una memoria de celdas sucias en su lugar.

En esta simulación, si hay más de 1 agente vivo y se encuentran, estos podrán “comunicarse” las celdas que conocen y las estaciones de carga visitadas, para así expandir su conocimiento del ambiente de manera colectiva. También un agente puede “ver” las celdas que tiene de vecinas en un radio de 2 celdas en cruz. Los agentes prefieren moverse a celdas sucias, luego a desconocidas,

Diseño de los agentes.

Agente	Objetivo	Capacidad Efectora	Percepción	Proactividad	Habilidad Social	Métricas de desempeño
Roomba	Conservar energía mientras busca celdas sucias	Moverse, limpiar celdas, comunicarse	- Escanear celdas en un rango de 2 a su alrededor - Detectar más Roombas - Detectar obstáculos - Detectar estaciones de carga - Detectar estado de celdas vecinas y recordar	- Prefiere moverse a celda sucia, luego desconocida, y finalmente aleatoria - Monitorea nivel de batería para tener suficiente y volver a base - Escoge estación de carga más cercana - Escoge moverse a celda sucia más cercana conocida después de carga	Comunica set de celdas y estaciones de carga conocidas a Roombas cercanas	Batería Celdas descubiertas Celdas limpiadas Roombas contactadas Viajes a estaciones

Arquitectura de subsunción de los agentes.



Características del ambiente.

El ambiente es inaccesible ya que un agente sólo puede detectar si las celdas contiguas en un esquema Von Neuman (4 puntos cardinales en lugar de los 8 vecinos, para que los agentes no brinquen en diagonales) está:

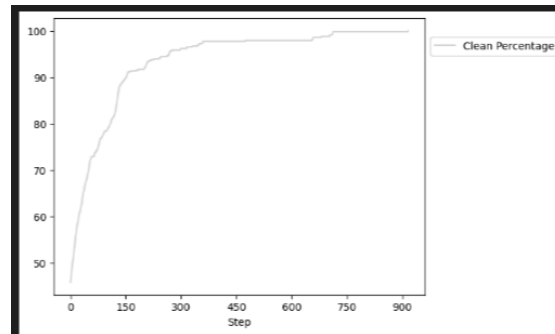
- Ocupada o desocupada
- Está sucia o limpia
- Es conocida o no
- Tiene estación de carga o vacía

Es no determinista ya que las posibilidades arraigadas a cada acción se tornan innumerables en cada Step, y los efectos se vuelven cascada. También es no determinista porque se genera con una semilla que se basa en los parámetros configurados para cada versión del modelo.

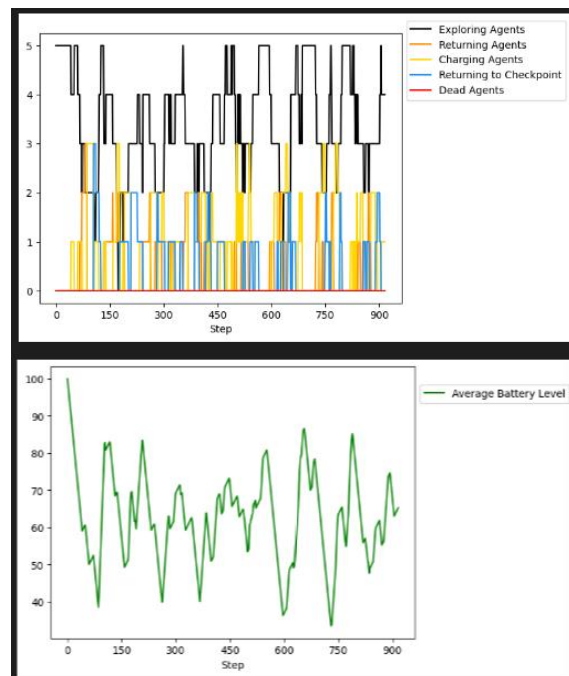
De igual forma, es episódico dado que cada Roomba define su siguiente acción a partir de condiciones actuales y en el momento de decisión únicamente. También es dinámico, ya que al limpiarse las celdas, esto hace que las "preferencias" de los Roombas cambien sobre a qué celda moverse, y al poder comunicarse, las preferencias de los demás agentes también cambian. Finalmente, el ambiente es discreto, ya que las posibilidades son contables y finitas.

Las estadísticas recolectadas en las simulaciones.

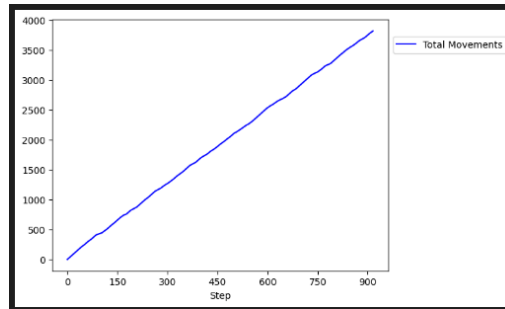
Ya que el objetivo de la simulación es desarrollar agentes que “limpien una habitación”, se almacenó en el DataCollector la cantidad de celdas limpias y sucias en cada Step, para así calcular el porcentaje de “limpieza” que se tiene en la habitación en cada Step.



Al estar continuamente actualizando y almacenando en cada agente Roomba su estado o actividad actual, pude graficar y guardar en el DataCollector cuántos agentes están “explorando”, “regresando a base”, “cargando”, o “inactivos” en cada Step de la simulación, así como un promedio del nivel promedio de batería en cada step de todos los agentes Roomba.



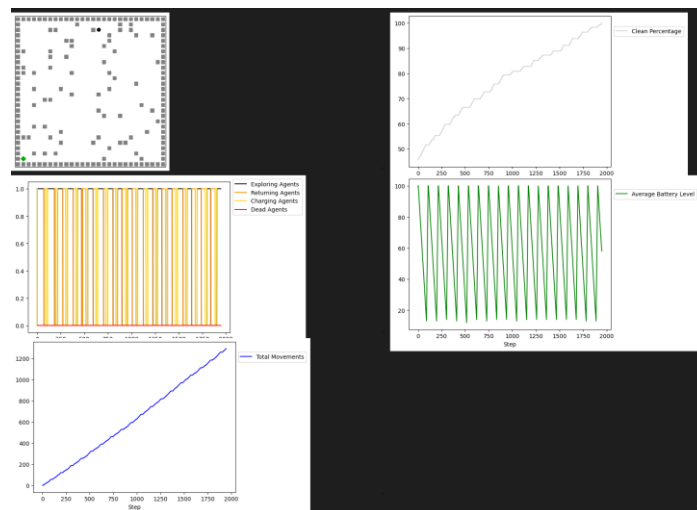
Finalmente, se almacenó también la cantidad de movimientos totales de todos los agentes en la simulación, es decir, cambios de posición en cada Step, para así determinar qué tantos movimientos requería una ejecución para limpiar la habitación.



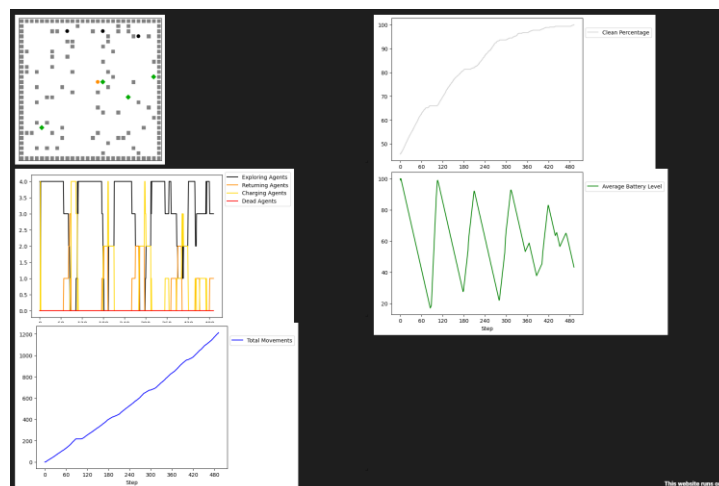
Por ejemplo, corriendo 1 simulación comparando Sim1 y Sim2 con configuración de:

- 80% celdas sucias
- 10% obstáculos
- 4 agentes en posiciones aleatorias para Sim2
- Seed 42
- 10000 Maximum steps

Sim1: completada en 1947 steps



Sim2: completada en 492 steps



Comparando ambos conjuntos de resultados, la segunda simulación sólo requirió en promedio 5 ciclos de carga, mientras que la individual 18. Esto representó un incremento en las vueltas que debía dar un agente, y tampoco le favorecía que para cruzar el mapa completo de esquina a esquina casi le tomara toda la batería que tenía disponible en dado momento para volver a cargar. No obstante, la cantidad de movimientos requerida para limpiar la habitación difirió en aproximadamente 200 movimientos, siendo la Sim1 quien tuvo ese 20% de incremento de trabajo. El comportamiento de porcentaje de la habitación en la Sim1 se asemeja a una función lineal, mientras que el comportamiento de la Sim2 parece una logarítmica. La programación inicial de los agentes anteriormente mostraba métricas de desempeño no tan buenas, y las gráficas de limpieza mostraban un comportamiento con muchos puntos silla en donde el agente no encontraba celdas sucias. Con el mecanismo de memoria de celdas sucias se solucionó este problema y así el agente va directamente a las faltantes.

Conclusiones.

La diferencia fue clara entre la Simulación 1 y Simulación 2. Al tener más agentes activos en la segunda, la limpieza fue mucho más veloz, y al poder comunicarse, se repetía menos la cantidad de celdas que re-visitaban en sus viajes y podían llegar más lejos en su exploración dado que reconocían las distintas estaciones de carga. No obstante, la cantidad de movimientos claramente incrementó ya que se cuentan por agente, y al tener más agentes, hubo más movimientos. También a mi computadora le costó más procesamiento, altamente notable cuando le puse 20 agentes a limpiar como experimento en un mapa totalmente sucio.

La arquitectura de subsunción fue la adecuada dada la importancia en la precedencia de las operaciones. Con versiones anteriores del flujo, a veces no terminaba de cargarse y se querían ir a explorar, o a veces estaban volviendo a casa y por ponerse a limpiar se olvidaban y se quedaban sin batería. Un punto para mejorar podría ser cómo se lleva a cabo el pathfinding a estaciones de carga, y en exploración ya que los algoritmos son algo rudimentarios y a fuerza bruta, haciendo la simulación computacionalmente compleja y algo ineficiente. No obstante, utilizar al menos BFS aprendido en clase, queues, y la función de unión entre sets fue particularmente útil para tareas muy específicas, y la versión modular de la máquina de estados permitía tener un mucho mejor control del flujo y acciones a llevar a cabo según las condiciones en la que se encuentra cada agente.